

CLASE 3 · UNIDAD 1 · SEMANA 3

Tokenización

De los bytes de UTF-8 al Byte-Pair Encoding:
construir un tokenizer desde cero

Inteligencia Artificial Generativa Avanzada y Sistemas Multi Agente

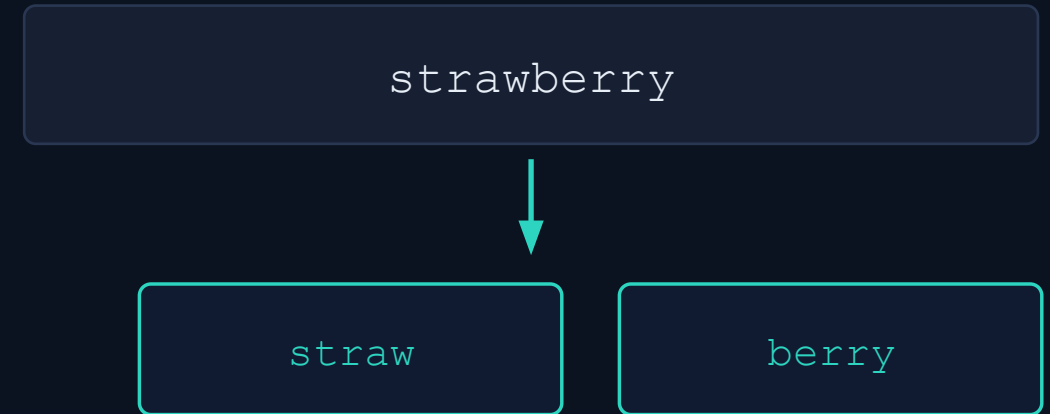
Prof. Francisco Traversaro · Licenciatura en Tecnología Digital, UTDT



Por qué importa la tokenización

La tokenización está en el origen de muchos comportamientos anómalos de los LLMs. Casos que parecen fallas de razonamiento son, en realidad, consecuencia de **cómo se segmenta el texto** en tokens.

Ejemplo. un LLM suele fallar al contar cuántas veces aparece una letra en una palabra (por ejemplo, las «r» en strawberry). No es un problema de conteo: el modelo no procesa caracteres, sino tokens, y dentro de un token las letras no son visibles.



el modelo ve 2 tokens, no 10 letras → no puede contar las «r»

La unidad mínima es el token, no el carácter. el tokenizer es un paso de preprocesamiento separado del modelo y determinista: fija con qué unidades opera el modelo. Estas limitaciones no se corrigen entrenando más; son propiedades de la representación.

Comportamientos que explica la tokenización

La siguiente lista reúne comportamientos observables de los LLMs. Todos tienen la misma raíz y se retoman, ya explicados, en la síntesis final.

Deletreo

No deletrea ni cuenta letras dentro de una palabra de forma confiable.

Aritmética

Se equivoca con números: se segmentan de formas inconsistentes.

Otros idiomas

Rinde peor fuera del inglés.

Espacios

Un espacio al final del texto puede alterar la respuesta.

Tokens poco frecuentes

Cadenas como SolidGoldMagikarp producen respuestas erráticas.

Formato

El mismo dato en YAML y en JSON consume distinta cantidad de tokens.

Origen común. ninguno es una falla de razonamiento: todos derivan de representar el texto como tokens, y no como caracteres.

Vocabulario (V) y longitud de secuencia (T)

Continuidad. en la clase de Transformers se trabajó a nivel de caracteres y la tokenización quedó pendiente. Es el tema de hoy.

Dos magnitudes. el vocabulario V (tamaño fijo para todo el modelo) y la longitud de secuencia T (varía con cada texto). La granularidad del tokenizer fija el compromiso entre ambas.

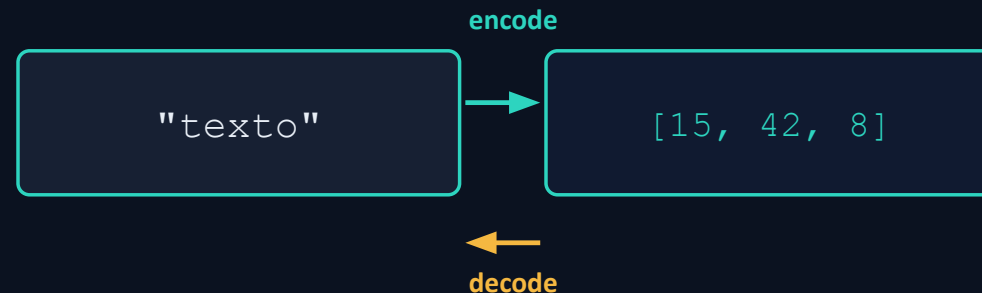
granularidad	vocabulario	secuencia	fuera de vocab.
por carácter	chico	larga	no
por palabra	enorme	corta	sí
subpalabra (BPE)	medio	media	no necesariamente

Objetivo: la fila del medio. subpalabra mediante BPE, que equilibra V y T. La longitud T que produce el tokenizer es la misma que procesa la atención: determina el uso del context window.

Qué vamos a construir

Objetivo. un tokenizer completo: entrena un vocabulario a partir de texto y provee encode (texto $\rightarrow$ ids) y decode (ids $\rightarrow$ texto).

Enfoque. se parte del caso más simple —los bytes crudos— y se agrega cada mecanismo cuando el anterior resulta insuficiente.



train *(una vez, aparte del modelo)*

corpus de texto $\rightarrow$ merges + vocab

El tokenizer se entrena por separado del modelo. su entrenamiento no involucra redes ni gradiente: es un conteo de frecuencias sobre un corpus. No debe confundirse con el entrenamiento del modelo de lenguaje.

Progresión de la clase

Cada etapa incorpora un mecanismo que resuelve una limitación de la anterior.



Alcance de hoy. las dos primeras etapas (Unicode → bytes) son el punto de partida; la compresión propiamente dicha comienza en el merge de BPE. (Bloque 1 llega hasta encode/decode; Bloque 2, de regex en adelante.)

Un string es una secuencia de code points

Representación interna. una computadora no procesa texto, sino números. Unicode asigna a cada carácter un entero: su code point.

Un string de Python es, en el fondo, una secuencia de code points. Hoy hay ~150.000 definidos en el estándar, y el número crece con cada versión.

```
ord("a")      # 97
ord("ñ")      # 241

chr(97)       # "a"
```

Opción tentadora: usar los code points como vocabulario. sería un id por carácter, sin fuera-de-vocabulario. Tiene un problema serio de tamaño y estabilidad, que la próxima lámina analiza.

Por qué no usar los code points crudos

La opción. vocabulario = todos los code points de Unicode. Ventaja: un id por carácter, sin fuera-de-vocabulario.

El problema. $V \approx 150.000$ y en aumento: el estándar cambia entre versiones. Es un vocabulario grande e inestable, y V es fijo para todo el modelo: la matriz de embeddings y la capa de salida escalan con V , y una gran cantidad de símbolos aparecería tan poco que quedaría sub-entrenada.

code points crudos

≈ 150.000 · crece con cada versión
matrices enormes · tokens raros sub-entrenados

buscamos



bytes de UTF-8

256 · fijo · universal

Requisito. un alfabeto pequeño, fijo y universal sobre el cual construir. Los bytes de la codificación UTF-8 lo cumplen.

Code point y bytes: dos representaciones distintas

Unicode y code point. Unicode es el estándar: el catálogo que asigna un número único a cada carácter. Un code point es ese número — una entrada del catálogo (se escribe U+XXXX).

El byte. un byte guarda un número de 0 a 255, y nada más. Un code point puede ser mucho mayor (el estándar reserva ~1.114.112 posiciones): no entra en un byte.

UTF-8. reescribe cada code point como 1 a 4 bytes. No comprime: cambia la unidad — así como el código Morse representa todo con 2 símbolos, UTF-8 lo hace con 256.

carácter	code point	bytes UTF-8	nº
A	65	[65]	1
ñ	241	[195, 177]	2
€	8364	[226, 130, 172]	3

~150.000 code points asignados hoy: los ideogramas CJK son ~90.000, más todos los alfabetos, símbolos y ~3.000 emoji. Un emoji ronda el 128.000.

Code point ≠ byte. el code point es el número; los bytes son su codificación. Por eso «ñ» tiene code point 241, pero se codifica como los bytes 195 y 177: UTF-8 transforma el número, no lo copia.

Cómo UTF-8 arma los bytes

El esquema. los primeros bits de cada byte forman un encabezado: indican cuántos bytes ocupa el carácter (0, 110, 1110, 11110) y cuáles son de continuación (10...). Los bits restantes contienen el code point.

Caso ASCII. si el code point es ≤ 127 , entra en un solo byte, que coincide con el número. Es la razón de que el texto en inglés use menos bytes.

```
ñ = U+00F1 = 241    (rango de 2 bytes)
plantilla: 110xxxxx 10xxxxxx
241 -> 110 00011 · 10 110001
      = 11000011 10110001 -> 195, 177
```

code point	byte 1	byte 2	byte 3	byte 4
U+0000-007F	0xxxxxxx	—	—	—
U+0080-07FF	110xxxxx	10xxxxxx	—	—
U+0800-FFFF	1110xxxx	10xxxxxx	10xxxxxx	—
U+10000-10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

En amarillo, la fila de la «ñ» (2 bytes). Los bits del encabezado marcan la longitud; las x llevan el code point.

Codificación reversible. de los bytes se reconstruye el texto exacto (decode). A cambio, un carácter puede ocupar varios bytes, de modo que la secuencia se alarga.

UTF-8 en Python: el vocabulario de 256 bytes

En Python. `str.encode("utf-8")` devuelve la secuencia de bytes; cada byte es un entero de 0 a 255.

El texto en inglés/ASCII usa 1 byte por carácter; los acentos, emojis y otros alfabetos usan más.

```
"hola".encode("utf-8")
# b'hola' -> [104,111,108,97]
# ASCII: 1 byte por carácter

"canCIÓN".encode("utf-8")
# la 'ó' ocupa 2 bytes
# 7 caracteres -> 8 bytes
```

Vocabulario base: 256. tokenizar byte a byte da un vocabulario fijo de 256, universal y sin fuera-de-vocabulario. Es el punto de partida —análogo al bigrama en la clase de Transformers—: correcto, pero mejorable.

Límite del vocabulario de 256: T crece demasiado

El problema. con un token por byte, las secuencias se vuelven muy largas: cada carácter no-ASCII ocupa 2–4 tokens, y aun el texto en inglés queda más largo de lo necesario.

Por qué importa. el costo de la atención crece con $O(T^2)$ (visto en la clase de Transformers): las secuencias largas son caras en cómputo, y un T mayor deja menos texto dentro del context window.

"gato" → 4 tokens

"canción" → 8 tokens (7 caracteres)

japonés / árabe → 3–4 tokens por carácter

atención $O(T^2)$ → un T mayor cuesta más cómputo

El compromiso. el vocabulario de 256 es pequeño y estable, pero produce un T alto. Hace falta comprimir la secuencia sin agrandar el vocabulario de forma indiscriminada. Eso es lo que hace BPE.

Code points, bytes y tokens: tres cosas distintas

Decisión 1 · la unidad base. no se usan los code points (~150.000) como vocabulario, sino los bytes de UTF-8 (256). UTF-8 no compite con Unicode: es la forma de escribir cada code point en bytes.

Decisión 2 · el vocabulario. BPE lo construye fusionando pares frecuentes de bytes en tokens (subpalabras y palabras); su tamaño se elige con vocab_size.

code points (~150.000 posibles)

UTF-8 · escribe cada uno en bytes



bytes (256) · base de BPE

BPE · merges por frecuencia



tokens (vocab elegido: 50k–200k)

GPT-2 ~50k → GPT-4 ~100k → GPT-4o ~200k: crecen a propósito.

El tamaño del vocabulario no es el problema. o200k_base tiene ~200.000 tokens —más que los ~150.000 code points— y no hay contradicción: son unidades distintas. BPE nunca opera sobre code points, siempre sobre los 256 bytes.

La idea de BPE

El mecanismo. sobre los bytes, BPE repite un paso: fusionar el par adyacente más frecuente en un token nuevo, haciendo crecer el vocabulario desde 256.

Por qué comprime. los pares frecuentes (como t+h en inglés) pasan a ser un solo token, de modo que las secuencias se acortan sin agrandar demasiado el vocabulario. El tamaño final de V lo define quien entrena el tokenizer: un merge = un token nuevo.

```
vocab inicial: 256 bytes (ids 0-255)
```

```
+ par más frecuente -> id 256  
+ siguiente par     -> id 257  
+ ...
```

```
vocab final: 256 + (nro. de merges)
```

Origen. BPE surgió como algoritmo de compresión (Gage, 1994) y se adaptó al NLP para tokenizar subpalabras (Sennrich et al., 2016). El principio es el mismo: reemplazar lo frecuente por un símbolo nuevo.

El procedimiento de BPE, en un ejemplo

El **procedimiento** (se repite hasta alcanzar el V deseado):

- 1 contar los pares adyacentes
- 2 elegir el más frecuente
- 3 definir un token nuevo
- 4 reemplazar todas sus ocurrencias
- 5 repetir

De 11 a 5 tokens con 3 merges. el tercer merge combina dos tokens ya fusionados (Z e Y); así BPE construye unidades progresivamente más largas, de bytes a subpalabras.

```
inicio:  a a a b d a a a b a c      (11)

1) par top "a a" (4×)  nuevo Z
   ->  Z a b d Z a b a c            (9)

2) par top "a b" (2×)  nuevo Y
   ->  Z Y d Z Y a c                (7)

3) par top "Z Y" (2×)  nuevo X
   ->  X d X a c                    (5)
```

Dos precisiones. (1) en el ejemplo se usan letras como bytes por legibilidad; en la práctica la base son los 256 bytes y el primer id nuevo es el 256 (la «a» es el byte 97). (2) el orden de los merges debe conservarse: es lo que encode utiliza después para reproducir la misma segmentación.

Qué produce el entrenamiento

El entrenamiento del tokenizer es un conteo de frecuencias — sin gradiente ni redes. Aplicado sobre un corpus, produce dos estructuras (con el ejemplo de la lámina anterior):

- **merges**: par → id nuevo, en el orden en que se aprendieron → lo usa encode.
- **vocab**: id → bytes → lo usa decode para reconstruir el texto.

```
# base: ids 0-255 = los 256 bytes
merges = {(97, 97): 256, # 'a','a' -> Z
          (97, 98): 257, # 'a','b' -> Y
          (256, 257): 258} # Z, Y -> X

vocab = {256: b'aa',
         257: b'ab',
         258: b'aaab'} # X = aa + ab
```

Hiperparámetro de diseño: vocab_size. fija el total de ids y, por lo tanto, la cantidad de merges ($\text{vocab_size} - 256$). Un vocabulario mayor acorta las secuencias, pero agranda las matrices del modelo y deja tokens poco frecuentes sub-entrenados.

Tres operaciones, un reglamento

train produce el reglamento una sola vez; **encode** y **decode** lo usan, siempre, sin modificarlo.

train(corpus) → **merges + vocab**

A partir del conteo de texto, decide qué pares fusionar. Una vez, offline. Sin gradiente.

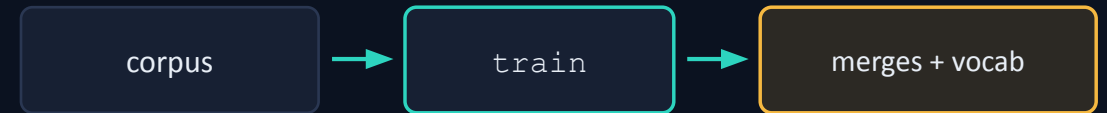
encode(texto) → **ids**

Aplica los merges ya aprendidos para comprimir texto nuevo. En cada entrada al modelo.

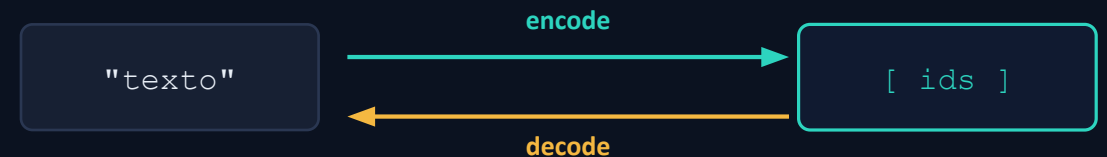
decode(ids) → **texto**

Busca cada id en vocab y reconstruye el texto. En cada salida del modelo.

① una vez · offline



② en cada request



ambos leen el reglamento de arriba

Diferencia de fondo. train descubre las reglas (por frecuencia); encode las aplica (en el orden aprendido, no por frecuencia); decode no aplica merges: solo consulta la tabla.

decode: de ids a texto

Reconstrucción por tabla. decode no aplica merges: las fusiones ya están contenidas en vocab (cada id guarda su secuencia de bytes). Es una consulta seguida de una reconstrucción.

Procedimiento. id → vocab[id] (bytes) → concatenar todos → decodificar UTF-8 → string.

► **En la notebook.** se implementa decode y se verifica que reconstruye el texto, id a id.

```
# ids -> bytes -> texto
def decode(ids):
    raw = b"".join(vocab[i] for i in ids)
    return raw.decode("utf-8",
                      errors="replace")

decode([258]) -> b'aaab' -> "aaab"
```

errors = replace. no toda secuencia de bytes es UTF-8 válido (un merge puede partir un carácter multibyte). Con esta opción, decode no genera error: inserta el carácter de reemplazo y continúa.

encode: de texto a ids

Orden de aplicación. encode aplica los merges en el orden en que se aprendieron —por prioridad de merge, no por frecuencia—. Es el punto donde suele haber confusión.

Procedimiento. texto → bytes → repetir: buscar el par presente cuyo merge se aprendió primero → fusionarlo → hasta que no quede ninguno aplicable.

► **En la notebook.** se implementa encode reutilizando la función de conteo de pares del entrenamiento.

```
encode("aaab") :  
  bytes          [97, 97, 97, 98]  
  merge (97,97)   ->  [256, 97, 98]  
  merge (97,98)   ->  [256, 257]  
  merge (256,257) ->  [258]  
  
"aaab"  ->  un solo token
```

Prioridad de merge, no frecuencia. encode elige el par cuyo merge está primero en la lista (por eso el entrenamiento conserva el orden). Descubrir las reglas es tarea del entrenamiento; encode solo las aplica.

Checkpoint: round-trip y compresión

CHECKPOINT — dos verificaciones que cierran la Parte 3.

1 · Round-trip exacto. `decode(encode(texto)) == texto`, para todo texto.
Garantiza que no se pierde información.

2 · Compresión. longitud en bytes frente a longitud en tokens: el ratio mide cuánto se acorta la secuencia, y aumenta con más merges.

► **En la notebook.** se corre el round-trip sobre un texto largo y se mide el ratio antes y después de entrenar.

```
# round-trip: siempre exacto
assert decode(encode(s)) == s

# compresión (ejemplo)
texto : 1000 bytes
tokens: 300 ids    -> 3.3x
```

Resultado. un tokenizer completo y reversible: entrena (por conteo), codifica (comprime) y decodifica (reconstruye) sin pérdida. Con esto ya puede alimentar al Transformer; resta adaptarlo a las condiciones del texto real, tema de las próximas partes.

El problema: fusiones a través de los límites

Sin restricciones. BPE fusiona cualquier par frecuente de bytes, sin distinguir letras, números, puntuación ni espacios. El efecto: una palabra se pega a lo que la rodea.

La misma palabra, muchos tokens. una palabra seguida de distintos signos (. ! ? ,) o con un espacio delante puede generar un token distinto por cada combinación frecuente. El modelo tiene que aprenderla por separado en cada caso, y el vocabulario se llena de casi-duplicados.

```
sin partir (BPE crudo):  
perro perro. perro! perro?  
perro, "perro" ...  
-> "perro" disperso en 10+ tokens
```

```
con partir primero:  
[perro] siempre un token  
[.] [!] [?] puntuación aparte  
-> "perro" se aprende una sola vez
```

La causa. BPE no tiene noción de «palabra» ni de «puntuación»: solo ve pares frecuentes de bytes. Hay que impedir las fusiones que cruzan esas fronteras — antes de correr BPE.

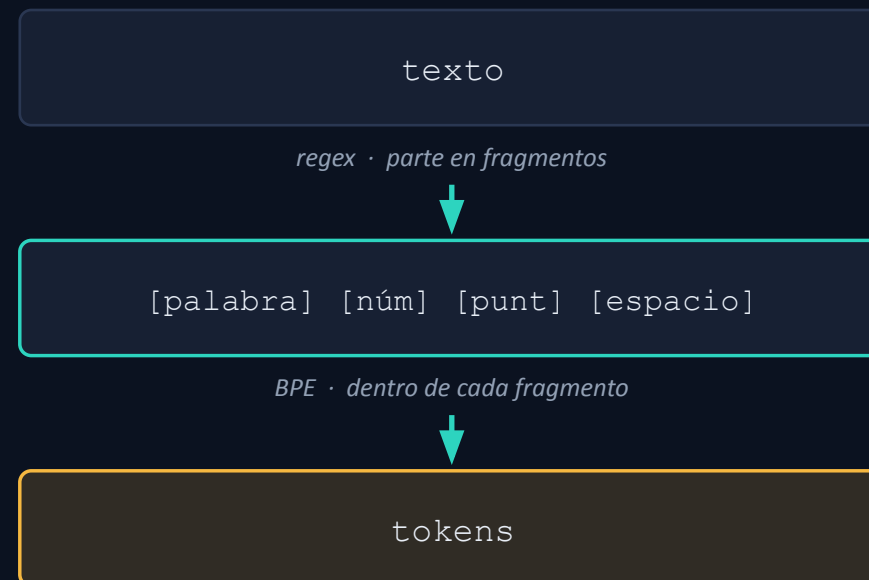
La solución: partir primero, después BPE

Un paso de segmentación antes de BPE. GPT-2 corta el texto con un patrón (regex) en fragmentos —palabras, números, puntuación, espacios— y BPE corre por separado dentro de cada fragmento. Los merges nunca cruzan un fragmento.

Consecuencia. una palabra queda igual sin importar qué signo la siga; los números no se fusionan con letras; la puntuación va aparte.

► **En la notebook.** train y encode primero parten con el regex y después aplican BPE fragmento por fragmento.

La regla. el regex define las fronteras; BPE solo trabaja dentro de cada fragmento, nunca a través de ellas.



El patrón de GPT-2, por dentro

Categorías separadas. el patrón (simplificado) corta el texto en contracciones, letras, números, puntuación y espacios — cada categoría por su cuenta, para que BPE no las mezcle.

Ejemplo. "Hola mundo 123!" → ["Hola", " mundo", " 123", " !"]

$\backslash p\{L\}$ = letra Unicode · $\backslash p\{N\}$ = número. Requiere la librería *regex* (no *re*).

```
's 't 're 've 'm 'll 'd # contracciones
?\p{L}+                # letras
?\p{N}+                # números
?[\^\s\p{L}\p{N}]+    # puntuación
\s+                    # espacios
```

El espacio se pega a la palabra siguiente (por el « ? » inicial). Por eso muchos tokens de GPT empiezan con espacio (" mundo"), y por eso un espacio final suelto puede alterar la segmentación — una de las rarezas de la apertura.

Qué cambió GPT-4

cl100k_base (GPT-4) refina el patrón de GPT-2 en tres puntos:

- **Contracciones sin distinguir mayúsculas.** antes 'S no coincidía con 's.
- **Números en grupos de 1 a 3 dígitos.** evita tokens de números enormes y mejora la aritmética.
- **Mejor manejo de espacios y saltos de línea**

El punto de fondo. estos patrones no se aprenden: se diseñan a mano, por prueba y error, y están pensados sobre todo para el inglés.

Reglas escritas a mano. el regex es la parte menos elegante del tokenizer: arbitraria y con sesgos (por ejemplo, hacia el inglés). Explica en parte varias de las rarezas de la apertura.

números en GPT-4:

GPT-2: "12345" -> irregular

GPT-4: "12345" -> "123" "45"
grupos de <= 3 dígitos
-> más consistente

Tokens especiales: fuera de BPE

Qué son. tokens que no representan texto natural, sino marcas de control: separan documentos, delimitan turnos de chat, señalan el fin de una generación.

Cómo entran al vocabulario. no se aprenden con BPE. Se agregan a mano, con un id reservado, por encima de los bytes y los merges.

```
vocab_size = 256 (bytes)
              + nro. de merges
              + nro. de especiales

GPT-2: 256 + 50.000 + 1
      (<|endoftext|>) = 50.257
```

El modelo tiene que aprenderlos. agregar un especial obliga a ampliar la matriz de embeddings y la capa de salida con una fila nueva: el token existe en el tokenizer, pero su significado lo aprende el modelo durante el entrenamiento.

Cómo los maneja encode

Se separan antes de BPE. encode busca primero las cadenas especiales, las reemplaza por su id reservado, y recién después corre BPE sobre los tramos de texto que quedan entre ellas. Un token especial nunca se parte en bytes.

```
"Hola<|endoftext|>Chau"
```

```
1. cortar por especiales
   ["Hola"] <|endoftext|> ["Chau"]

2. BPE solo en los tramos de texto
   [BPE]      id 50256      [BPE]

-> [..ids..] [50256] [..ids..]
```

encode tiene que «permitirlos» explícitamente. si el texto de un usuario contiene la cadena literal <|endoftext|>, interpretarla como token de control (y no como texto) es un riesgo de seguridad. Por eso las librerías obligan a habilitarlos (allowed_special).

Para qué se usan

Delimitar documentos. `<|endoftext|>` (el clásico, desde GPT-2) separa textos independientes en el corpus.

Marcar turnos de chat (ChatML). los modelos de chat envuelven cada mensaje con `<|im_start|>` ... `<|im_end|>` (derecha). El modelo genera hasta `<|im_end|>` y ahí frena.

Relleno / FIM (GPT-4). `<|fim_prefix|>`, `<|fim_middle|>`, `<|fim_suffix|>` completan en el medio de un texto (código).

```
<|im_start|>user  
Hola<|im_end|>  
<|im_start|>assistant
```

El rol (system / user / assistant) va como texto común después de `<|im_start|>`: no es un token especial, se tokeniza con BPE. Solo los delimitadores son especiales.

`<|im_start|>` / `<|im_end|>` son de los modelos de chat, posteriores a GPT-2 (que solo tenía `<|endoftext|>`). No son mágicos: son tokens de control agregados al vocabulario para delimitar los turnos.

tiktoken: la librería de OpenAI

Qué es. la librería open-source de OpenAI que implementa el BPE byte-level que construimos. Es lo real detrás del encode/decode de GPT; núcleo en Rust → muy rápida.

Qué NO hace: entrenar. solo infiere — trae los vocabularios ya entrenados y hace encode/decode.

encodings que trae:

r50k_base	-> GPT-2, GPT-3	(~50k)
cl100k_base	-> GPT-3.5, GPT-4	(~100k)
o200k_base	-> GPT-4o	(~200k)

Es solo la mitad de «inferencia» de lo que vimos. la mitad del train OpenAI no la liberó como open source.

Ejercicio opcional: armá tu tokenizer de GPT-4

El gap. tiktoken no entrena. Para ver por dentro esa mitad, el ejercicio de minbpe (Karpathy) reconstruye el tokenizer entero desde cero.

Autochequeo. que su encode/decode coincidan exactamente con tiktoken.

Para casa · opcional · no evaluado. Repo: [karpathy/minbpe](#).

qué construyen:

```
BasicTokenizer (bytes)
  -> RegexTokenizer (split GPT-4)
  -> reproducir GPT-4 (cl100k)
      vía tiktoken
  -> tokens especiales
```

Es la pieza que falta del lado de OpenAI: entrena el BPE byte-level para ver lo que tiktoken ya trae hecho.

Algoritmos vs librerías

No confundir el método con la herramienta. un algoritmo decide cómo se eligen los subwords. Una librería lo implementa (y agrega entrenamiento, normalización, velocidad).

3 ALGORITMOS

BPE fusiona los pares más frecuentes

WordPiece fusiona por otro criterio (BERT)

Unigram poda probabilística (lo vemos ahora)

3 LIBRERÍAS

tiktoken BPE · solo infiere

SentencePiece BPE o Unigram · entrena

HuggingFace tokenizers las tres · entrena

Una librería elige un algoritmo — no van 1 a 1. WordPiece es un algoritmo (no una librería), y SentencePiece puede correr BPE o Unigram.

Unigram: el modelo

Cada subword tiene una probabilidad. el vocabulario le asigna a cada token x una probabilidad $p(x)$ — cuán «usado» es como unidad. Todas las $p(x)$ suman 1.

«Unigram» = tokens independientes. la probabilidad de una segmentación es el producto de las $p(x)$ de sus tokens (cada token no depende de los vecinos — por eso «uni-gram»).

Un texto se puede segmentar de muchas formas; el modelo prefiere la de mayor producto.

"hola"

segmentación : probabilidad

[hola] $p(\text{hola})$

[ho][la] $p(\text{ho}) \cdot p(\text{la})$

[h][o][la] $p(\text{h}) \cdot p(\text{o}) \cdot p(\text{la})$

La probabilidad reemplaza a la frecuencia. en BPE manda qué par aparece más; en Unigram, qué segmentación es más probable según el modelo.

Unigram: entrenar es podar (top-down)

El objetivo: la verosimilitud. la verosimilitud (likelihood) mide qué tan bien el vocabulario explica el corpus: la probabilidad que el modelo le asigna a todo el texto de entrenamiento. Más alta = mejor. Entrenar = maximizarla con el vocab_size fijo.

BPE construye; Unigram talla.

BPE (bottom-up): arranca de los símbolos base y fusiona hacia arriba.

Unigram (top-down): arranca con un vocabulario grande y poda los que menos aportan.

```
semilla: vocabulario grande
| (1) ajustar las p(x) <- EM
| (2) podar lo que menos aporta
| repetir
v
vocab final (vocab_size)
```

Dos movimientos que se repiten: ajustar las probabilidades (EM) y podar el vocabulario. Las próximas dos láminas desarmarán cada uno.

Unigram: ajustar las probabilidades (EM)

EM = Expectation-Maximization (esperanza—maximización): un método iterativo para estimar las $p(x)$ cuando no sabemos de antemano cómo se segmentó el corpus.

El problema del huevo y la gallina. para estimar $p(x)$ necesitaría saber cómo se segmenta el texto; pero para segmentar necesito las $p(x)$. EM lo rompe alternando dos pasos, y en cada vuelta el modelo explica mejor el corpus hasta estabilizarse.

E — Expectation (esperanza)

con las $p(x)$ actuales, calcula la frecuencia esperada de cada subword (pesando cada segmentación por su probabilidad).



M — Maximization (maximización)

recalcula cada $p(x) = \text{frecuencia esperada de } x \div \text{total}$.

EM estima lo que no se observa. no vemos la «segmentación verdadera» del corpus; EM la infiere y va afinando las probabilidades de forma consistente.

Unigram: podar el vocabulario

Achicar el vocabulario sin romper el ajuste. con las $p(x)$ ya estimadas (EM), Unigram decide qué descartar:

- Para cada subword, mide cuánto baja la verosimilitud si lo saca. Si el texto se re-segmenta bien sin él, es redundante.
- Elimina el X% que menos aporta.
- Nunca elimina los caracteres sueltos: hay que poder segmentar cualquier texto.

ciclo completo:

```
semilla grande
  |
  | EM + poda    (repetir)
  v
vocab_size objetivo

(los caracteres nunca se podan)
```

Se queda con los subwords que más explican. en vez de construir de a un merge, Unigram descarta lo que aporta poco, quedándose con lo que mejor comprime el corpus.

Unigram: segmentar con Viterbi

Segmentar un texto nuevo = elegir la de mayor producto de $p(x)$, entre todas las segmentaciones posibles. Probarlas todas es inviable: crecen exponencialmente con el largo.

Viterbi: el atajo. un algoritmo de programación dinámica (reusa sub-resultados en vez de recalcular). Recorre el texto de izquierda a derecha y guarda solo el mejor camino hasta cada posición → mejor segmentación global sin enumerarlas todas.

```
"hola"  
[hola]      p(hola)  
[ho][la]    p(ho) · p(la)  
[h][o][la]  ...
```

```
Viterbi -> la de mayor producto  
           (eficiente)
```

Bonus. al ser probabilístico, Unigram puede muestrear segmentaciones distintas del mismo texto al entrenar (aumento de datos). BPE no tiene eso.

Greedy vs óptimo. BPE replica sus merges en orden fijo (un camino, rápido); Unigram evalúa alternativas y elige la más probable.

SentencePiece: la librería

Qué es. el toolkit de tokenización de Google. La librería de la familia no-OpenAI: Llama, Mistral, T5, Gemma. Entrena e infiere.

Corre BPE o Unigram (su default es Unigram). Llama 2 lo usa con BPE; T5 con Unigram. La librería es la misma; el algoritmo adentro varía.

Todo integrado: normalización, manejo de espacios, tokens especiales, byte_fallback.

SentencePiece (toolkit)

BPE

Unigram

(default)

entrena + infiere · normaliza · byte_fallback

Es una librería, no un algoritmo. lo que la separa de tiktoken no es el algoritmo (puede ser el mismo BPE), sino cómo trata el texto → próxima lámina.

SentencePiece: sobre code points, no bytes

La decisión de base. corre BPE/Unigram sobre code points de Unicode, no sobre bytes. Es el camino inverso al de GPT/tiktoken (retoma la bisagra de code points vs bytes).

byte_fallback. a los caracteres fuera del vocabulario los manda a bytes UTF-8 solo como respaldo → sin fuera-de-vocabulario.

Texto crudo, sin asumir palabras → language-independent: sirve para japonés o chino, que no separan con espacios.

```
el espacio es un símbolo: _
```

```
"Hola mundo"
```

```
-> _Hola _mundo
```

```
el espacio queda DENTRO del token
```

```
-> reconstrucción exacta (reversible)
```

Mismo algoritmo posible (BPE), otra unidad base (code points). ese es el «cómo y por qué» que separa el tokenizer de Llama 2 del de GPT.

WordPiece: el algoritmo de BERT

El tercer algoritmo. bottom-up como BPE, pero fusiona con otro criterio: en vez del par más frecuente, elige el que más aumenta la verosimilitud del corpus.

En la práctica, el score de un par a, b es $\approx \text{frec}(ab) / (\text{frec}(a) \cdot \text{frec}(b))$: alto cuando a y b aparecen juntos más de lo esperable por azar $\rightarrow$ prioriza pares que «van juntos», no solo los frecuentes.

```
## marca continuación de palabra:
```

```
playing -> play ##ing
```

```
encoding: greedy longest-match
```

```
toma el subword más largo del  
vocabulario que encaje, y sigue.
```

Primo de BPE, criterio distinto. es un algoritmo (el de BERT), no una librería; lo implementan el código original de BERT y HuggingFace tokenizers.

Quién usa qué

familia	arquitectura	algoritmo	librería
GPT (2/3/4)	decoder	BPE byte-level	tiktoken
BERT	encoder	WordPiece	BERT / HF tokenizers
T5	enc-dec	Unigram	SentencePiece
Llama 2 · Mistral	decoder	BPE (code points)	SentencePiece
Llama 3	decoder	BPE byte-level	— (estilo tiktoken)

La tendencia se unifica. los modelos nuevos (Llama 3) migran al BPE byte-level estilo tiktoken. La distinción por familias ayuda, pero las fronteras se corren — y en el lab, HuggingFace AutoTokenizer carga y compara todos estos.

Volvemos a la lista: cada rareza, explicada

La lista de la apertura, ahora con la causa de tokenización de cada comportamiento.

Deletreo

las letras no son visibles dentro de un token → no puede contarlas ni deletrear.

Aritmética

los números se parten inconsistentes (123|4 vs 1|234): sin representación estable.

Otros idiomas

tokenizer entrenado sobre todo en inglés → más tokens por palabra, menos datos.

Espacios

el espacio va pegado a la palabra; un espacio final suelto rompe el patrón aprendido.

Tokens poco frecuentes

SolidGoldMagikarp: token visto por el tokenizer, casi no entrenado (→).

Formato

el mismo dato cuesta distintos tokens: JSON «desperdicia» más que YAML.

Ninguna es magia. cada comportamiento de la apertura se sigue de una decisión concreta de la tokenización — la que fuimos construyendo toda la clase.

SolidGoldMagikarp: el token fantasma

El caso. ciertas cadenas raras (como SolidGoldMagikarp, un usuario de Reddit) hacían que GPT respondiera de forma errática: las repetía mal, cambiaba de tema, se comportaba raro.

La causa. el tokenizer se entrena sobre su propio corpus, distinto del corpus con el que se entrena el modelo. Lo que es frecuente para uno puede estar ausente para el otro.

Resultado. un token válido en el vocabulario cuya fila de embedding nunca se ajustó → comportamiento impredecible.

```
corpus del TOKENIZER (mucho Reddit)
  "SolidGoldMagikarp" es frecuente
  -> le toca UN TOKEN propio (id N)
```

```
corpus del MODELO (sin esos datos)
  ese token casi no aparece
  -> su embedding NUNCA se entrena
```

```
id N: token real, vector sin ajustar
```

Entrenar el tokenizer ≠ entrenar el modelo. es la distinción de la Parte 3, ahora con consecuencias visibles: un vocabulario puede tener tokens que el modelo nunca aprendió a usar.

¿Cuándo se usa un tokenizer en la práctica?

Casi siempre está escondido. al consumir un LLM por API, la tokenización corre del lado del servidor y no la tocás. Se puede trabajar años con LLMs sin llamar a un tokenizer — lo aprendemos para entender el comportamiento, no porque se use a diario.

Cuándo sí lo usás, directamente:

- **Contar tokens** — estimar costo, verificar la ventana de contexto, y partir en chunks para RAG. El uso más común: `tiktoken.encode(texto)`.
- **Entrenar o fine-tunear** — ahí el texto → ids es la entrada misma del modelo.
- **Truncar** — para modelos con límite duro (un clasificador BERT de 512, un modelo de embeddings).
- **Debuggear** — mirar cómo se parte un texto explica muchos errores raros.
- **Control fino** — prohibir o forzar tokens (`logit_bias`) necesita sus ids. Nicho, pero existe.

Consumís API → casi nunca. Construís infraestructura → sí. pipelines de datos, RAG a escala, entrenamiento y optimización de costos: ahí el tokenizer deja de estar escondido.

Cierre: lo que construimos

```
texto -> code points -> bytes UTF-8 -> BPE (merges) -> encode/decode  
-> regex -> tokens especiales -> tokenizers reales -> rarezas explicadas
```

La idea de fondo. la tokenización es un paso de preprocesamiento separado, determinista y elegido a mano, que fija la unidad mínima con la que opera el modelo. No es glamorosa, pero es el compromiso necesario: lo ideal sería alimentar el modelo con bytes crudos, pero dispararía T y el costo $O(T^2)$ de la atención.

Lo que sigue. cada token entra al modelo como un vector (embedding): cómo se construye ese token $\rightarrow$ vector es el próximo tema.

De token a significado. ya tenemos el texto convertido en ids. Darle a cada id un vector que capture significado es donde arranca lo que viene.

